TECHNICAL DOCUMENTATION · PROGRESS REPORT

AI Agent Project Manager

A multi-agent system that turns project documentation into a validated, sprint-ready delivery plan.

DOCUMENT TYPE	**System architecture, agent catalogue and progress report**
PREPARED FOR	**Team Lead**
VERSION	**1.0**
DATE	**October 12, 2026**
STACK	**Python · CrewAI · Ollama (local LLM) · Pydantic · FastAPI · Docker**

Contents

1. Executive Summary . . . 3
2. Problem, Goal and Scope . . . 4
3. System Architecture . . . 5
4. The Agent Team . . . 7
5. Orchestrator and Decision Logic . . . 10
6. Deterministic Planning Layer . . . 11
7. Validation, Repair and Quality Gates . . . 12
8. Reliability Engineering for Small Local Models . . . 13
9. Data Model and Output Artefacts . . . 14
10. Project Assistant (Chat and REST API) . . . 14
11. Configuration and Deployment . . . 15
12. Results Achieved . . . 16
13. Testing and Quality Assurance . . . 18
14. Known Limitations and Observations . . . 18
Appendix A — Repository Structure . . . 19

1. Executive Summary

The **AI Agent Project Manager** reads a project-documentation file (Markdown) and automatically produces a complete, validated delivery plan: requirements, epics, tasks with hour estimates, dependencies, priorities, risks, a sprint schedule, milestones and per-member task assignments. It is built with **CrewAI** on top of **local LLMs served by Ollama**, so project data never leaves the machine, and it is driven by a central **Orchestrator** that decides which specialised agent runs next.

> **Core design principle — "LLMs propose, Python decides"**
> Agents never touch the project state. Each one receives a small slice of the document or plan and returns a strictly-typed *proposal*. All identifiers, scheduling, capacity limits, role matching and validation are executed by deterministic Python code. This is what makes the output reliable even with small (3B–7B) local models.

Status at a glance

9 specialised agents	**13** orchestrator actions	**39** validation rule codes	**29** automated test cases	**2** sample projects planned

What has been delivered

- **End-to-end planning pipeline** from a Markdown document to a validated `final_project_plan.json` plus ten separate artefacts.
- **Nine specialised agents** (requirements, epics, tasks, dependencies, priorities, sprints, assignments, risks, semantic validation), each with its own role, goal, prompt and output schema.
- **Resilient orchestration:** state-driven decision loop, transactional state merging, retry-with-feedback, per-step deterministic fallbacks, loop guards and checkpoint/resume.
- **Authoritative validation:** 39 deterministic rules gate completion; an LLM semantic review adds advisory notes only.
- **Project Assistant:** a scoped Q&A chatbot (CLI, web page and REST API) that answers only from the generated plan, in Arabic or English.
- **Deployment package:** Dockerfile and a production Docker Compose file (Ollama + web app) with API-key protection.
- **Verified on two projects** (a customer-support platform and a clinic-management system), both passing final validation with zero errors.

Headline results

On the clinic-management project the system produced **35 requirements, 8 epics, 48 tasks (318 h), 45 dependencies, 5 risks, 4 sprints and 4 milestones in about 18 minutes** using the 7B model, with **0 validation errors**. A 3B-parameter model completed a smaller plan in about one minute, which makes it useful for fast iteration, while the 7B model gives noticeably richer dependency analysis (30 dependencies versus 1 on the same sample document). Section 12 gives the full comparison and Section 14 lists known limitations honestly.

2. Problem, Goal and Scope

Turning a project brief into an executable plan is repetitive manual work: extracting requirements, grouping them into epics, estimating tasks, finding blocking relations, balancing workload and checking that nothing is inconsistent. The goal of this project is to automate that work with a team of cooperating AI agents while guaranteeing structural correctness.

	Description
Input	A Markdown project document with ## sections (overview, objectives, scope, out of scope, users, functional and non-functional requirements, constraints, team, timeline, milestones, dependencies, risks, MVP definition), plus `config/team.json` (8 team members with roles, skills and weekly capacity) and `config/settings.json`.
Output	Requirements, epics, tasks, dependencies, priorities, sprints, assignments, risks, milestones, validation report, per-member workload and an execution log; all merged into `final_project_plan.json`.
In scope	Planning from documentation, sprint scheduling within team capacity, automatic validation and repair, project Q&A over the generated plan.
Out of scope (current version)	Execution tracking after planning, integrations with Jira/Trello/Azure DevOps, multi-project management, authentication beyond a shared API key.

3. System Architecture

The system is organised around one rule: the Orchestrator owns the loop, the State owns the truth, and agents are stateless advisers. Figure 1 shows the main components and how data flows between them.

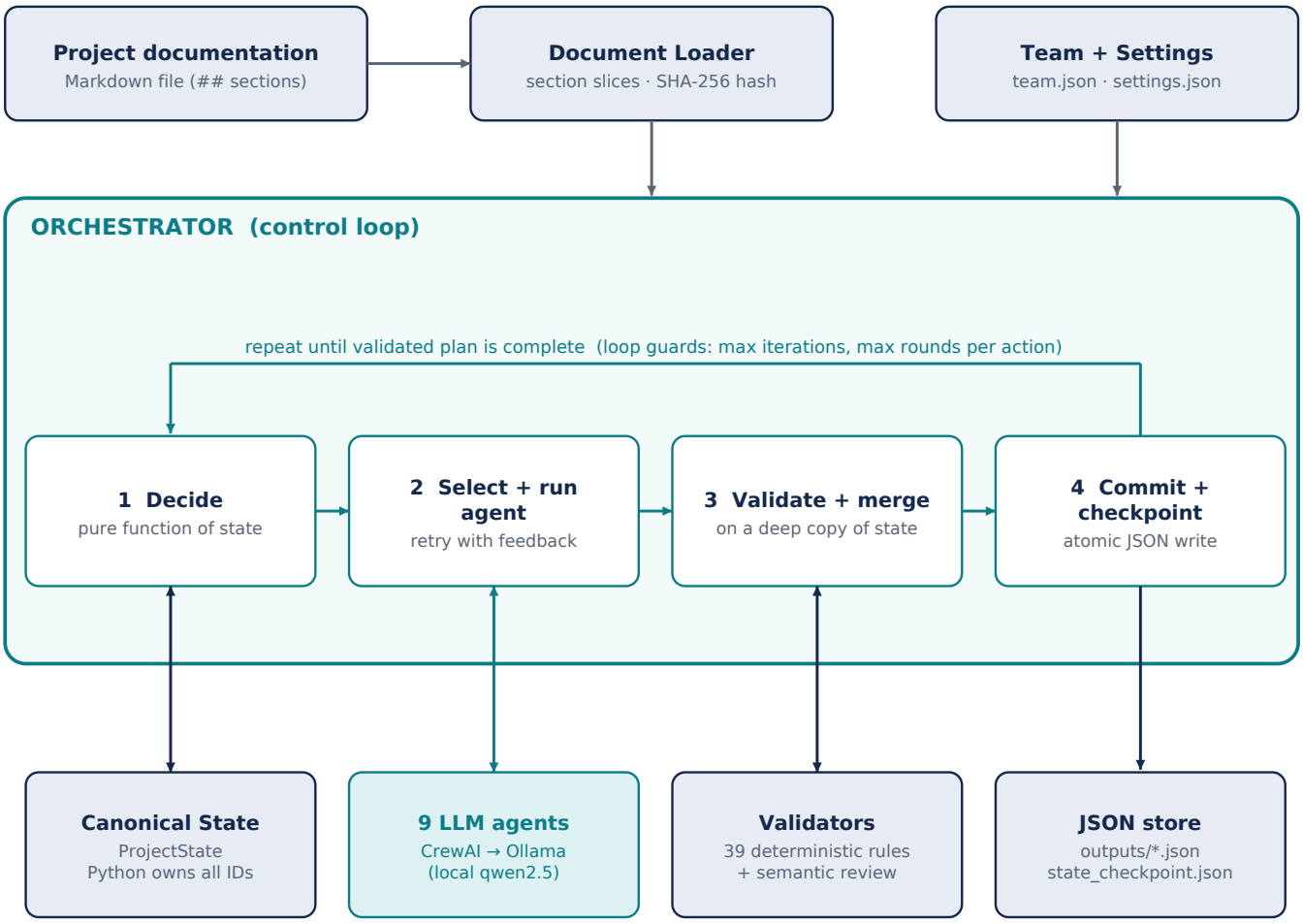


Figure 1 — High-level architecture. Each loop iteration inspects the canonical state, picks the next action, runs the matching agent, validates the proposal on a copy of the state, and only then commits.

Component map

Component	Location	Responsibility
Entry point	`main.py`	CLI (`--doc --test --mock --resume --model --output-dir`); pre-flight Ollama checks and model warm-up; prints a run summary. Exit codes: 0 = validated plan, 1 = runtime error, 2 = validation failed.
Document loader	`services/document_loader.py`	Splits the Markdown into sections and serves small, targeted slices (default cap 3,500 chars) to each agent; stores a SHA-256 hash so a checkpoint is only resumed against the same document.
CrewAI runner	`crew/runner.py`	Wraps each agent as a cached CrewAI `Agent` and runs every call as a single-task `Crew` against `ollama/<model>` (temperature 0.1, 300 s timeout, watchdog +30 s).
Agents	`agents/`	Nine specialised agents sharing a `BaseAgent` that builds prompts, appends rejection feedback, extracts JSON and validates it against a Pydantic schema.
Decision engine	`orchestrator/decisions.py`	A pure function *(state, team, settings) → next action*; satisfied steps are skipped, so agents never run just because they are "next in a chain".
Orchestrator	`orchestrator/orchestrator.py`	Main loop, agent selection, transactional merge, retries, fallbacks, repair, finalisation and execution logging.
Canonical state	`orchestrator/state.py`	`ProjectState` holds every entity; Python generates all IDs (REQ-, EPIC-, TASK-, SPRINT-, RISK-, MS-) and normalises any ID an LLM returns.
Planning logic	`orchestrator/planning.py`	Deterministic sprint scheduler, task assigner and heuristic dependency generator.
Validators	`validators/`	Authoritative deterministic validator (39 rule codes) and an advisory semantic validator that wraps the Project Validator agent.
Services	`services/`	Robust JSON extraction, atomic JSON store, Ollama health checks, logging, error hierarchy, dependency-graph cycle detection, offline mock LLM.
Project Assistant	`chat.py, server.py`	Scoped Q&A over the final plan: CLI, browser page and REST API (FastAPI).

Pipeline phases

Work flows through four phases. Teal boxes are LLM-backed agents that *propose*; navy boxes are deterministic Python that *decides*.

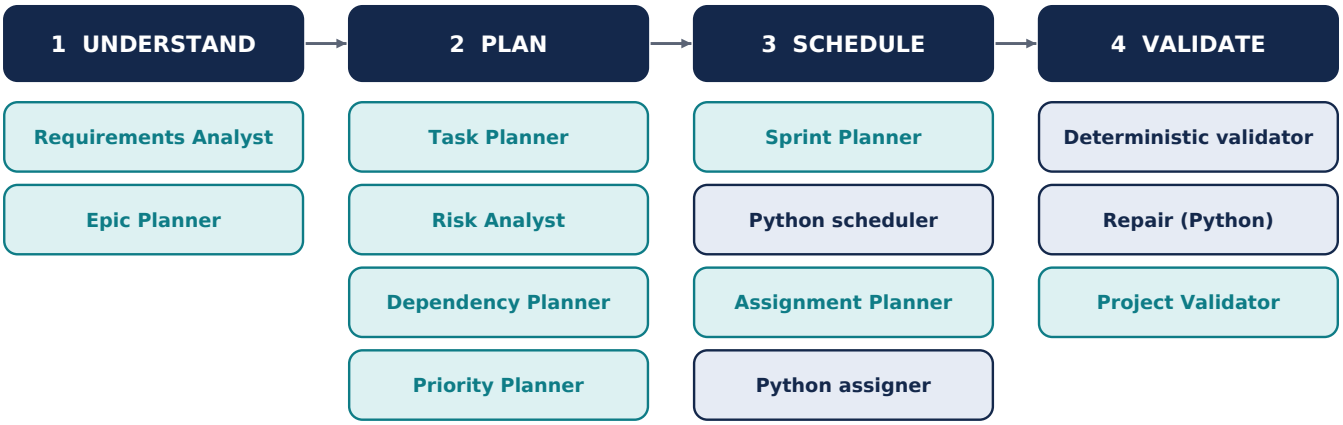


Figure 2 — Planning phases and the component responsible for each step.

4. The Agent Team

There are **nine specialised agents**. Each is a CrewAI persona (role, goal, backstory) combined with a prompt builder and a strict output schema. Delegation between agents is disabled; coordination happens only through the Orchestrator. Two agents (Requirements Analyst and Task Planner) are invoked in several modes.

#	Agent	One-line role	Invoked in	If it fails
1	**Requirements Analyst**	Extracts project facts and every functional / non-functional requirement from the document.	analyze_project, extract_requirements (×2)	Fatal: planning cannot continue
2	**Epic Planner**	Groups requirements into 5–8 delivery epics.	create_epics	Fatal
3	**Task Planner**	Breaks each epic into small, estimable tasks; fills gaps for missing task types.	create_tasks (per epic + gap mode)	Step skipped; repair or gap-fill later
4	**Dependency Planner**	Finds real blocking relations between tasks.	create_dependencies	Heuristic dependencies generated in Python (stage order inside each epic)
5	**Priority Planner**	Assigns critical / high / medium / low per task.	set_priorities	Defaults from task-level priority
6	**Risk Analyst**	Identifies 5–8 delivery risks with mitigations.	detect_risks	Step skipped
7	**Sprint Planner**	Defines sprint goals, milestone names and epic start sprints.	plan_sprints	Python plans sprints with default goals
8	**Team Assignment Planner**	Matches tasks to the best-fit member when several share a role.	assign_tasks	Python assigns by least load
9	**Project Validator**	Semantic review of the finished plan (advisory).	semantic_review	Review skipped

4.1 Agent profiles

1 · Requirements Analyst	FATAL ON FAILURE
Role / goal	Senior business analyst. Extract project facts and requirements from the documentation accurately and concisely; never invent a requirement that is not in the document.
Modes	**project** (summary, objective, duration in weeks, actors, MVP scope, out-of-scope, assumptions, business priority) · **functional** (one item per capability) · **non-functional + constraints**.
Input	Targeted document sections only (e.g. overview, objectives, scope, users, MVP, timeline; or the functional-requirements section).
Output	`ProjectAnalysis` or `RequirementsOutput` (title ≤ 6–8 words, description ≤ 15 words, priority, kind).
Guardrails	Lists capped at 10 items; actors must be user roles; titles and descriptions length-limited to keep small models inside their output budget.

2 · Epic Planner	FATAL ON FAILURE
Role / goal	Agile coach. Group requirements into a small set of coherent delivery epics covering foundation, features, quality and release.
Input	Project summary plus the list of requirement IDs, titles and kinds.
Output	`EpicsOutput`: 5 to *max_epics* (8) epics, each with name, description, priority and linked requirement IDs.
Guardrails	Every functional requirement must belong to at least one epic; `requirement_ids` may only come from the supplied list; foundation, QA and release epics are added when the requirements call for them.

3 · Task Planner	SKIPPED ON FAILURE
Role / goal	Tech lead. Break an epic into concrete, verifiable engineering tasks with realistic hour estimates.
Modes	**epic mode** — one call per epic, 3 to 8 tasks, each 2–16 h, with an hour budget derived from team capacity · **gap mode** — adds 2–4 tasks for every task type that the team can execute but the plan lacks (e.g. no QA or design tasks).
Output	`TasksOutput`: title (starts with a verb), type (backend, frontend, database, design, qa, devops, documentation), description, hours, priority.
Guardrails	No IDs in the output — Python assigns TASK-NNN. Gap mode must reference epic IDs from the supplied list.

4 · Dependency Planner	HEURISTIC FALLBACK
Role / goal	Delivery manager. Record only dependencies that truly block work, and only between *tasks* — never epics.
Input	Compact task list: `ID [type] title (epic)`.
Output	`DependenciesOutput`: `task_id` cannot start until `depends_on` is finished, with a ≤ 10-word reason.
Guardrails	At most 2 dependencies per task; typical blockers only (design → frontend, schema → backend, API → integration, implementation → QA, QA → deployment); no cycles. Python rejects unknown IDs, self-dependencies, duplicates and cycles.

5 · Priority Planner	DEFAULT PRIORITIES
Role / goal	Product owner. Rank tasks by business value and delivery risk to protect the MVP scope and deadline.
Input	Business priority statement plus the tasks that still lack a priority.
Output	`PrioritiesOutput`: critical (MVP cannot work without it / blocks many tasks), high (core MVP), medium, low (polish).
Guardrails	Runs at most twice; leftovers are filled with task-level defaults. Priorities are recomputed after dependencies change.

6 · Risk Analyst	SKIPPED ON FAILURE
Role / goal	Risk manager focused on schedule, integration, quality and scope risks of short MVP projects.
Input	Project summary, the document's risk and dependency notes, and the epic list.
Output	`RisksOutput`: 5 to 8 risks with likelihood and impact (low/medium/high), mitigation (≤ 20 words) and linked epic IDs.
Guardrails	Epic references restricted to the supplied IDs; the number of risks is capped by *max_risks* (10).

7 · Sprint Planner	PYTHON FALLBACK
Role / goal	Scrum master. Sequence foundation first, features next, hardening and release last.
Input	Timeline from the document and the epic list with task counts and hours.
Output	`SprintPlanOutput`: a goal (≤ 15 words) and milestone name for each sprint, plus the sprint in which each epic should *start*.
Guardrails	The agent only *suggests*. Placement of every task is done by the Python scheduler (Section 6), which respects dependencies and capacity. If the plan changes later, sprints are re-planned deterministically, keeping the goals and hints.

8 · Team Assignment Planner	PYTHON FALLBACK
Role / goal	Resource manager. Match tasks to the best-suited member by role and skills while balancing weekly workload; never invent people.
Input	Members (skills, current load, capacity) and the sprint's tasks, each with an *eligible* member list.
Output	`AssignmentsOutput`: `task_id → member_id` suggestions.
Guardrails	Called only when a task's role has more than one eligible member; otherwise assignment is purely deterministic and costs no LLM call. A suggestion is honoured only if the role matches, capacity remains and the load stays balanced.

9 · Project Validator	ADVISORY
Role / goal	PMO reviewer. Spot semantic problems that structural checks cannot see: sprint goals that do not match their tasks, missing important work, unrealistic ordering.
Input	A compact digest of each sprint's goal and task titles.
Output	`SemanticReview`: verdict (*pass* or *concerns*) and up to 5 short notes.
Guardrails	**Advisory only.** It can add notes but can never override the deterministic validator or block completion.

5. Orchestrator and Decision Logic

The Orchestrator implements a single loop: **inspect state → decide next action → select agent → run agent → validate output → merge into canonical state (transactionally) → consistency check → retry / repair → repeat**. Completion is gated by a full deterministic validation.

5.1 Decision priority

`decide_next_action` walks the following checklist in order and returns the first action whose precondition is unmet.

Order	Condition	Action (agent)
1	No project facts yet	analyze_project (Requirements Analyst)
2	Functional / non-functional requirements missing	extract_requirements (Requirements Analyst)
3	Requirements exist but no epics	create_epics (Epic Planner)
4	An epic has fewer than 2 tasks	create_tasks, one call per epic (Task Planner)
5	Team has a role with no matching task type	create_tasks in gap mode (Task Planner)
6	Risks not analysed	detect_risks (Risk Analyst)
7	More than one task and no dependency analysis	create_dependencies (Dependency Planner)
8	Tasks without priority	set_priorities (Priority Planner)
9	Tasks not scheduled, or a sprint has unassigned tasks	plan_sprints (Sprint Planner) / assign_tasks (Assignment Planner)
10	State changed since last validation	validate_plan (Python)
11	Validation failed and repair budget remains (2 rounds)	repair_plan (Python)
12	Validation passed, no semantic review yet	semantic_review (Project Validator)
13	Everything complete	generate_summary → finalise and write outputs

5.2 Safe state updates

- Every agent proposal is merged into a **deep copy** of the state. The copy is run through the deterministic validator (*partial* stage); it is committed and checkpointed only if the merge succeeds and the consistency check passes.
- If the proposal is rejected (schema violation, unknown reference, inconsistent state), the agent is called again with the exact rejection reason appended to the prompt — up to **3 attempts**.
- Timeouts and transient LLM errors are retried as well; an unreachable Ollama server or a missing model aborts immediately with an actionable message.
- Each commit increments a state **revision** and writes `state_checkpoint.json` atomically, enabling `--resume`.

5.3 Failure policy per action

Action	After 3 failed attempts
analyze_project, extract_requirements, create_epics	**Fatal** — an OrchestrationError stops the run (further planning would be meaningless).
create_tasks	Step marked *skipped*; the repair loop re-queues under-filled epics later.
detect_risks	Marked skipped.
create_dependencies	Heuristic dependency pairs are generated in Python, priorities recomputed, step marked *fallback*.
set_priorities	Task-level priorities are used as defaults.
plan_sprints	Python-only planning with the goals/hints already known.
assign_tasks	Deterministic assignment (least-loaded eligible member).
semantic_review	Marked skipped; does not affect completion.

5.4 Loop guards

Guard	Setting	Value
Maximum loop iterations	`max_iterations`	80
Maximum times the same action may be selected	`max_action_rounds`	6
Attempts per agent call	`max_agent_attempts`	3
Repair rounds after failed validation	`max_repair_rounds`	2
LLM timeout per call	`llm_timeout_seconds`	300 s (+30 s watchdog)

6. Deterministic Planning Layer

Everything that must be *correct* rather than *creative* is done in Python (`orchestrator/planning.py`).

6.1 Sprint scheduler

- Places every task in exactly one sprint, picking the next ready task by **priority rank**, then priority level, then epic start hint.
- A task can never start before the sprint of any task that blocks it (dependency order).
- Capacity is enforced per **role pool** with a 90 % utilisation target (falling back to 100 %), plus a soft per-sprint total (average × 1.15) to level load across sprints.
- QA tasks cannot start before sprint 2 (`type_min_sprint`).
- If a task fits nowhere it is placed in the last sprint and the validator reports the capacity violation — constraints are never silently broken.

6.2 Task-to-role mapping and team

Task type	Executing role	Team members (weekly capacity)
backend, database	Backend Developer	TEAM-002, TEAM-003 (30 h each)
frontend	Frontend Developer	TEAM-004, TEAM-005 (30 h each)
design	UI/UX Designer	TEAM-006 (30 h)
qa	QA Engineer	TEAM-007 (30 h)
devops	DevOps Engineer	TEAM-008 (30 h)
documentation	Project Manager	TEAM-001 (10 h)

Total team capacity is 220 hours per one-week sprint. Skills in `team.json` (e.g. “react, dashboard, charts”) guide the Assignment Planner when two members share a role.

6.3 Assignment and heuristic dependencies

- **Assignment:** an LLM suggestion is accepted only when the member has the right role, enough remaining capacity and is not more than 40 % of a sprint’s capacity above the least-loaded peer; otherwise the least-loaded eligible member is chosen. When every eligible role has a single member, no LLM call is made.
- **Heuristic dependencies** (fallback) work inside each epic: a task depends on the last task of the nearest earlier stage in the order design → database → backend → frontend → QA.

7. Validation, Repair and Quality Gates

Validation runs at two stages. The *partial* stage runs after every merge to catch inconsistent state early; the *final* stage adds completeness and a serialisation round-trip. A plan is marked `complete` only if the final stage passes and the validated revision equals the current revision; otherwise the status is `failed_validation` and a `draft_project_plan.json` is written instead.

Category	Examples of rule codes (39 in total)
Identity & schema	DUPLICATE_ID, MISSING_ID, BAD_ID_FORMAT, MISSING_FIELD, INVALID_ENUM, INVALID_STATUS, SCHEMA_ROUNDTRIP
References	INVALID_REFERENCE, INVALID_SPRINT_REF, INVALID_ASSIGNMENT_TASK, INVALID_ASSIGNMENT_MEMBER, INVALID_DEPENDENCY_REF
Dependencies	SELF_DEPENDENCY, DUPLICATE_DEPENDENCY, CIRCULAR_DEPENDENCY, DEPENDENCY_ORDER
Estimates & priorities	INVALID_ESTIMATE, INVALID_PRIORITY, MISSING_PRIORITY, DUPLICATE_PRIORITY, PRIORITY_MISMATCH
Sprints & capacity	SPRINT_COUNT, EMPTY_SPRINT, TASK_NOT_IN_SPRINT, TASK_IN_MULTIPLE_SPRINTS, SPRINT_CAPACITY, ROLE_CAPACITY, PLANNED_HOURS_MISMATCH
Assignments	TASK_NOT_ASSIGNED, DUPLICATE_ASSIGNMENT, ROLE_MISMATCH, ASSIGNMENT_SPRINT_MISMATCH, MEMBER_OVERLOAD
Completeness	EPIC_WITHOUT_TASKS, EPIC_TOO_FEW_TASKS, REQUIREMENT_UNCOVERED, EMPTY_SECTION, MISSING_PROJECT

7.1 Automatic repair

When final validation fails, `repair_plan` maps error codes to fixes without calling an LLM:

- **Dependency errors** → invalid, self-referencing and duplicate edges are dropped; cycles are broken by removing the closing edge; sprints are re-scheduled.
- **Sprint / capacity errors** → deterministic re-planning and full re-assignment.
- **Epic with too few tasks** → the epic is re-queued so the Task Planner runs again, and dependency analysis is re-queued.
- **Priority errors** → priorities are rebuilt from defaults.
- **Assignment errors** → invalid assignments are dropped and rebuilt.

After two unsuccessful repair rounds the run finishes as *failed_validation* rather than shipping an unverified plan.

> **Semantic review is advisory by design**
> The Project Validator (LLM) may flag, for example, that a sprint goal does not match its tasks. These notes are stored in `validation.json` but never change the pass/fail result, because a small model's opinion must not override verified rules.

8. Reliability Engineering for Small Local Models

Local 3B–7B models are fast and private but make predictable mistakes. The system was designed around those mistakes; the offline mock LLM (`services/mock_llm.py`) deliberately reproduces several of them so the guards are exercised in tests.

Typical model failure	Protection implemented
Wraps JSON in markdown fences, adds prose, trailing commas, or truncates the answer	`extract_json` strips fences, finds the first balanced object, removes trailing commas and repairs truncated output.
Returns numbers where strings are expected, a string instead of a list, priority synonyms ("urgent", "P1")	Lenient proposal types normalise values; extra keys are ignored. Canonical state models are strict (`extra=forbid`).
Invents or mixes up IDs (e.g. an EPIC ID used as a TASK ID)	Python generates all IDs; incoming references are normalised and checked, unknown references are rejected with feedback.
Produces circular or self-referencing dependencies	Graph cycle detection on every merge; repair removes the closing edge.
Assigns work to a person who does not exist or has the wrong role	Eligible-member lists are provided in the prompt and enforced again in code.
Times out or hangs	300 s call timeout, watchdog thread, single worker, bounded retries, then a per-action fallback.
Switches to another language (e.g. Chinese in chat answers)	CJK detector: retry once, then strip.
Endless re-selection of the same step	Per-action round cap and global iteration cap abort with a clear error.
Resuming against a different document	Checkpoint stores the document hash; mismatch aborts the resume.
Half-written output after a crash	Atomic writes (temp file + `os.replace`).

9. Data Model and Output Artefacts

All entities live in the canonical `ProjectState`. Identifiers follow a fixed format enforced by the validator.

Entity	ID format	Main fields
Requirement	REQ-001	title, description, priority, kind (functional / non_functional / constraint)
Epic	EPIC-001	name, description, priority, linked requirement IDs
Task	TASK-001	title, type, description, estimated hours, priority, epic, status (planned / in_progress / blocked / done)
Dependency	—	task_id, depends_on, type (blocks), reason
Sprint	SPRINT-001	number, goal, task IDs, planned hours, capacity hours
Assignment	—	task_id, member_id, sprint_id, reason
Risk	RISK-001	title, description, likelihood, impact, mitigation, epic IDs
Milestone	MS-001	name, sprint, week, criteria

Files written to the output directory

File	Content
`final_project_plan.json`	The complete validated plan: project, requirements, epics, tasks, dependencies, priorities, sprints, assignments, risks, milestones, validation, team and per-member workload. (`draft_project_plan.json` if validation failed.)
`requirements / epics / tasks / dependencies / priorities / sprints / assignments / risks .json`	One file per entity type for easy consumption.
`validation.json`	Pass/fail, error and warning counts, issue list, semantic verdict and notes.
`execution_log.json`	Every orchestrator step: iteration, action, reason, agent, status, seconds, detail and state revision; plus total time, LLM calls and model.
`state_checkpoint.json`	Full state snapshot for `--resume`.
`run.log`	Human-readable log with tags ORCHESTRATOR, AGENT, RETRY, VALIDATION, STATE UPDATE, ERROR.

10. Project Assistant (Chat and REST API)

Once a plan exists, the **Project Assistant** lets anyone ask questions such as “Who owns TASK-012?”, “What is in sprint 3?” or the same in Arabic. The plan is compressed into a compact one-line-per-item context (tasks with sprint, owner and dependencies, sprints, team, risks, milestones) and the model is instructed to answer only from it.

Three guard layers keep the assistant on topic

- **Layer 1 — deterministic:** plan IDs (TASK-007, SPRINT-2 …) or project keywords (English and Arabic) mark the question as in scope with no LLM call.
- **Layer 2 — classifier:** otherwise a tiny YES/NO call (max 3 tokens) decides; any doubt means refuse.
- **Layer 3 — answer prompt:** the model must reply `OUT_OF_SCOPE` for unrelated questions, which is converted into a polite refusal in the user's language.

Interfaces

Interface	Details
CLI	`python chat.py --plan outputs/final_project_plan.json --model qwen2.5:7b`
Browser	Single-page chat served at `/`; supports right-to-left text; stores a session ID in the browser.
REST	`GET /health` → status, model, plan loaded. `POST /ask` with `{question, session_id}` → `{answer}`.
Security	Optional API key via `x-api-key` header with constant-time comparison; question length limited to 1,000 characters.
Sessions	Up to 200 in-memory conversations (oldest evicted); the last 6 messages are kept as history.

11. Configuration and Deployment

11.1 Key settings (`config/settings.json`)

Setting	Value	Meaning
model	qwen2.5:3b (file) · qwen2.5:7b-instruct (code default)	Ollama model; override with `--model` or env `PM_MODEL`
temperature	0.1	Low randomness for structured output
max_context_chars	3,500	Cap on the document slice sent to an agent
default_duration_weeks / sprint_length_weeks	4 / 1	Defaults when the document does not state them
utilization_target	0.9	Target role-pool utilisation before falling back to 100 %
max_epics / max_tasks_per_epic / min_tasks_per_epic	8 / 8 / 2	Plan-size limits
max_risks / max_dependencies_per_task	10 / 3	Plan-size limits
min / max_estimate_hours	1 / 24	Accepted task estimate range
type_min_sprint	{qa: 2}	QA work cannot start before sprint 2

11.2 Running the system

Purpose	Command
Install	`pip install -r requirements.txt` (crewai, pydantic, httpx, fastapi, uvicorn)
Offline smoke test (no LLM)	`python main.py --test --mock`
Plan a project	`python main.py --doc path/to/project.md --model qwen2.5:7b-instruct`
Resume after interruption	`python main.py --doc path/to/project.md --resume`
Unit tests	`python3 -m unittest discover -s tests -v`
Ask questions (CLI)	`python chat.py`
Web / API server	`uvicorn server:app --host 0.0.0.0 --port 8000`

11.3 Docker

The `Dockerfile` (Python 3.11-slim, build tools for CrewAI native dependencies) runs the self-test by default. `docker-compose.prod.yml` starts **two services on one server**: `ollama` (with a persistent model volume and an optional Nvidia GPU block) and web (FastAPI/uvicorn on port 8000). The web service reads the plan read-only from `./plan/final_project_plan.json`, requires an `API_KEY` environment variable and defaults to the `qwen2.5:7b` model. Deployment steps are documented in the compose file header.

12. Results Achieved

The system was executed on two sample projects. All figures below come from the generated execution logs and plan files in the project archive.

	Run A — Support platform	Run B — Support platform	Run C — Clinic system
Model	qwen2.5:7b-instruct	qwen2.5:3b (resumed from a checkpoint)	qwen2.5:7b
Duration	3,542 s (≈ 59 min)	59 s	1,088 s (≈ 18 min)
LLM calls / steps	24 / 24	8 / 10	24 / 24
Tasks · hours	48 · 344 h	27 · 166 h	48 · 318 h
Dependencies	30	1	45
Validation	PASS · 0 errors · 3 warnings	PASS · 0 errors · 0 warnings	PASS · 0 errors · 4 warnings
Semantic review	concerns (4 notes)	concerns (4 notes)	concerns (3 notes)

Run A figures are taken from its execution log and run log (sprint summary S1–S4); only these two files from Run A are in the project archive, so its task and hour totals are derived from that summary.

12.1 Project 2: Clinic Appointment & Patient Management System

Source: `clinic_project.md` — a 4-week MVP for receptionists, doctors and a clinic manager. Generated plan: **35 requirements, 8 epics, 48 tasks, 45 dependencies, 5 risks, 4 sprints, 4 milestones**.

Sprint	Goal	Tasks	Planned h / capacity
1	Implement foundations	9	54 / 220
2	Develop core features	13	88 / 220
3	Add status handling and UI	16	90 / 220
4	QA and deployment	10	86 / 220

Epic	Priority
EPIC-001 User Authentication	high
EPIC-002 Patient Management	high
EPIC-003 Doctor Management	high
EPIC-004 Appointment Booking	high
EPIC-005 Search and Filtering	high
EPIC-006 Backend and Database	medium
EPIC-007 Frontend and Deployment	medium
EPIC-008 Quality Assurance	low

Identified risks: authentication delays (high/high), booking-conflict bugs, schema changes, integration delays, QA bottleneck.

12.2 Per-member workload (clinic plan, hours per sprint)

Member	S1	S2	S3	S4	Total
Backend Developer 1	20	28	24	14	86
Backend Developer 2	26	22	24	16	88
Frontend Developer 1	0	10	8	8	26
Frontend Developer 2	0	10	12	24	46
UI/UX Designer	8	10	0	0	18
QA Engineer	0	8	16	18	42
DevOps Engineer	0	0	6	6	12
Project Manager	0	0	0	0	0

12.3 Project 1: Customer Support & Service Management Platform

Source: `data/test_project_documentation.md` (16 sections). The 7B run produced 48 tasks over four sprints (S1 = 48 h / 7 tasks, S2 = 98 h / 15, S3 = 94 h / 13, S4 = 104 h / 13) with 30 dependencies. The final saved 3B plan has 23 requirements, 5 epics, 27 tasks, 6 risks and 4 milestones (M1 authentication and schema → M4 deployed and accepted by the support team), aligned with the timeline in the source document.

12.4 Observations

- **Model size matters for relational reasoning.** On the same document the 7B model produced 30 dependencies versus 1 from the 3B model; the 3B model is excellent for smoke tests and quick iterations.
- **Dependency analysis is the slowest step on 7B** (860 s in Run A, 180 s in Run C) and hit two 300 s timeouts in Run A before succeeding on the third attempt — the retry mechanism worked as designed.
- **Self-healing in practice:** in Run C the Sprint Planner returned invalid JSON twice; the rejection reason was fed back and the third attempt was accepted.
- **Validation is strict but practical:** all runs ended with 0 errors; the four clinic warnings are REQUIREMENT_UNCOVERED (see Section 14).

13. Testing and Quality Assurance

The repository contains **29 automated test cases** that run without Ollama by using the offline mock LLM.

Suite	Cases	What it covers
`tests/test_pipeline.py`	16	Full mock run writes all outputs and passes validation; Python owns IDs; epic without tasks fails safely; Dependency Planner failure triggers the fallback; validator rules (duplicate ID, EPIC-as-TASK ID, unknown epic, cycle, missing/duplicated sprint tasks, dependency order, capacity/estimate/member errors, invalid enum/status); JSON extraction (fences, prose, trailing commas, truncated output, garbage).
`tests/test_chat.py`	9	Compact context; ID/keyword scope shortcut; off-topic refusal in English and Arabic; keyword-bait caught by the second layer; Chinese output retried then stripped.
`tests/test_server.py`	4	Health endpoint and page; API-key enforcement; answers and refusals; empty-question rejection.

Test-case counts were taken from the test files. The suites run with `python3 -m unittest discover -s tests -v`.

Real-model verification is done with `python main.py --test`, which runs the bundled sample document and checks that all eleven expected output files exist and that the key plan sections are non-empty.

14. Known Limitations and Observations

Documented openly so the team can prioritise next steps.

Area	Observation	Impact
Semantic review	All three runs ended with verdict “concerns” (e.g. sprint goals judged too broad). The reviewer is a small model and advisory only.	Low — does not affect validity, but sprint goals could be more specific.
Dependency quality	The 3B model produced a single dependency for 27 tasks; the 7B model produced 30–45.	Medium — use ≥ 7B for production plans.

Area	Observation	Impact
Speed	7B planning took 18–59 minutes depending on the run; the Dependency Planner dominates and can hit the 300 s timeout.	Medium — GPU server or longer timeout recommended.
Requirement coverage	In the clinic plan four functional requirements (doctor schedule view, dashboard, basic reporting, staff management) are not linked to any epic (warnings).	Medium — tasks may still exist, but traceability is incomplete.
Milestone naming	In the clinic plan milestones are named only “M1…M4”; the support plan reuses the full milestone text from the document.	Low — cosmetic.
Utilisation	Plans use about 36 % (clinic) and 19–39 % (support) of the 880 h available over four sprints.	Info — estimates may be optimistic or the team under-used.
Project Manager load	The PM role receives almost no tasks (documentation tasks only).	Low.
Configuration	`settings.json` uses qwen2.5:3b while the code default and Docker Compose use 7B models.	Low — align before release.
Packaging	The shared archive contains `.env`, compiled `__pycache__` folders and output logs.	Low — exclude from distribution; `.dockerignore` already excludes .env.
Scope	Planning only; no progress tracking, no external tool integrations, shared API key only.	Planned work.

Suggested next steps

The following are recommendations for discussion, not completed work.

- Run planning on a GPU server with the 7B (or larger) model and compare plan quality across models on a small benchmark of project documents.
- Add a traceability check that every functional requirement is covered by at least one task, and a repair step for REQUIREMENT_UNCOVERED.
- Improve milestone and sprint-goal generation (give the Sprint Planner the task titles per epic) to reduce semantic-review concerns.
- Export the plan to tools the team already uses (Jira, Trello, CSV/Excel) and add progress tracking after planning.
- Add per-user authentication and a plan-upload endpoint to the web service.
- Run the 29 existing tests in CI on every commit.

Appendix A — Repository Structure

```
AI_project_manager/
├── main.py                  CLI entry point
├── chat.py                  Project Assistant (CLI + guard layers)
├── server.py                FastAPI web + REST API
├── agents/                  9 specialised agents + BaseAgent
│   ├── requirements_agent.py  epic_agent.py  task_agent.py
│   ├── dependency_agent.py    priority_agent.py  risk_agent.py
│   └── sprint_agent.py  assignment_agent.py  validator_agent.py
├── orchestrator/
│   ├── orchestrator.py      control loop, retries, fallbacks, repair
│   ├── decisions.py         next-action decision function
│   ├── state.py             canonical ProjectState, ID ownership
│   └── planning.py          scheduler, assigner, heuristic deps
├── validators/              deterministic_validator.py, semantic_validator.py
├── crew/runner.py           CrewAI + Ollama execution layer
├── models/                  Pydantic models and enums
├── services/                loader, JSON tools, storage, Ollama, graph, logging, mock LLM
├── config/                  settings.json/.py, team.json
├── data/                    sample project documentation
├── tests/                   test_pipeline.py, test_chat.py, test_server.py
├── outputs/  outputs_clinic/  generated plans and logs
└── Dockerfile  docker-compose.prod.yml  requirements.txt
```

Appendix B — Glossary

Term	Meaning
Agent	A CrewAI persona (role, goal, backstory) plus prompt builder and output schema that returns a validated proposal.
Orchestrator	The control loop that decides which action and agent run next and owns all state changes.
Canonical state	The single source of truth (`ProjectState`) from which all output files are generated.
Proposal	Structured output of an agent; never applied directly to the state.
Epic	A group of related requirements delivered together.
Sprint	A one-week planning period with a fixed team capacity.
Checkpoint	Saved state revision that allows an interrupted run to resume.
Ollama	Local runtime that serves open-source LLMs (here qwen2.5) on the team’s own hardware.